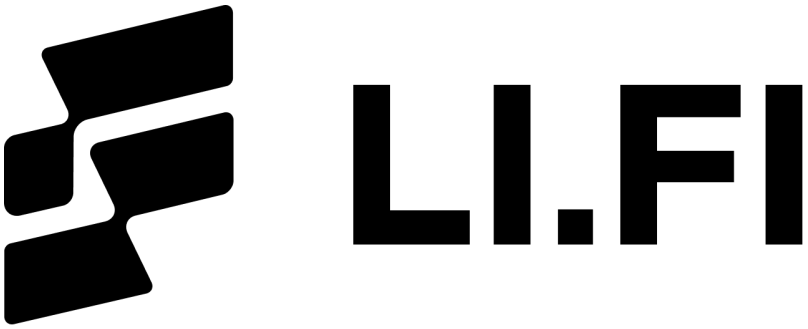

LI.FI Security Review

ERC7821LIFI - Catapultar

Security Researcher

Sujith Somraaj (somraajsujith@gmail.com)

Report prepared by: Sujith Somraaj

July 21, 2026

Contents

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over $170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols.

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

- solidity/src/libs/ERC7821LIFI.sol

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

High leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.

Medium global losses <10% or losses to only a subset of users, but still unacceptable.

Low losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

High almost certain to happen, easy to perform, or not easy but highly incentivized

Medium only conditionally possible or incentivized, but still relatively likely

Low requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Critical Must fix as soon as possible (if already deployed)

High Must fix (before deployment if not already deployed)

Medium Should fix

Low Could fix

5 Executive Summary

Over the course of 6 hours in total, LI.FI engaged with the researcher to audit the contracts described in section 3 of this document ("scope").

This is an differential review, and hence the scope is limited only to the changes introduced in the audit commit.

In this period of time a total of 7 issues were found.

Project Summary	
Project Name	LI.FI
Repository	lifinance/catapultar
Commit	48ac2493a
Audit Timeline	July 14, 2026 - July 21, 2026
Methods	Manual Review
Documentation	High
Test Coverage	Medium-High

Issues Found	
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	4
Gas Optimizations	0
Informational	3
Total Issues	**7**

6 Findings

6.1 Low Risk

6.1.1 Arbitrary callees can forge `EstimateGasStarved` and force estimation failure

Context: ERC7821LIFI.sol#L187-L189

Description: In `EstimateGas` / `RaiseRevertEstimate` mode, before the gas-based starvation check runs, a failed call whose returndata is exactly `0x24` bytes and begins with the `EstimateGasStarved(uint256)` selector `0xaf3228d9` is treated as a nested estimation frame's own starvation verdict and re-raised verbatim. Nothing verifies that the returndata actually originated from a nested `Catapultar` frame.

Because estimation exists precisely to call arbitrary third-party targets, any called contract can `revert` with `abi.encodeWithSelector(0xaf3228d9, X)` for an attacker-chosen `X`; the frame then bubbles it unchanged to the top-level estimator, aborting estimation, injecting an attacker-controlled `gasLeft`, and bypassing the flag-2 skip path.

Recommendation: Only treat `EstimateGasStarved` as trusted when it comes from a Catapultar self-call child, e.g. `to == address(this)`. For arbitrary external callees, classify the failure using the local gas threshold like any other revert.

LI.FI: Acknowledged. This is intended and desired. Each time you go down the stack, the 1/64 gas that is not forwarded causes problems for "top-level" estimation:

A called contract can only estimate given the gas it has been given. If this gas is spent to the entirety, when the OOG moved up the stack and we observe the failure, we check for gas spend BUT under the constraint that we have to add our 1/64 allocation. For a gas capturing contract that is 1 frame below, the strength of the check is half as strong. Allowing a called contract to do the computation on their end will allow them to surface more accurate gas estimation to us.

Researcher: Acknowledged.

6.1.2 Fixed starvation threshold does not hold on high-gas EVM chains

Context: ERC7821LIFI.sol#L30-L35

Description: `EstimateGas` treats a failed call as gas-starved only if the Catapultar frame has less than `262,144` gas remaining.

That threshold is derived from Ethereum-sized gas limits. Under EIP-150, an OOG child call leaves roughly `1/64` of the parent frame's gas behind, so `262,144 * 64 = 16,777,216`.

This works only when the relevant transaction or frame gas budget is capped around that range. It does not hold on high-throughput EVM chains with much larger limits. For example, MegaETH documents transaction compute gas limits up to `200,000,000`. In that environment, a child call can run out of gas while the parent still has well over `262,144` gas left. Catapultar would then treat the failure as a normal skippable revert instead of a starvation signal, allowing gas estimation to settle on a value where the expensive path is never priced.

Recommendation: Do not hardcode a threshold based on Ethereum's gas cap if Catapultar is intended to be multichain. Make the threshold chain-specific, derive it from the maximum frame or transaction gas used during estimation, or cap SDK-driven estimation to the range where the threshold is valid.

LI.FI: Acknowledged. The intention is to get the same addresses on every chain; We need to use the lowest common denominator which is Ethereum.

I am aware that this is bad practise since such a value can be changed in the future. While unlikely, if lowered it would completely nuke this deployment.

However, the value 262,144 is constant and inserted into the code at compile time. This estimation trick is mainly an off-chain trick using code overwrite, so one could find and replace 262,144 with the appropriate number on a chain by chain basis.

Researcher: Acknowledged.

6.1.3 Estimation modes can be used in signed production transactions

Context: ERC7821LIFI.sol#L82-L83

Description: `ERC7821LIFI` registers both `EstimateGas` and `RaiseRevertEstimate` as supported execution modes. The TypeScript SDK uses these modes for gas preflight through an unsigned self-call estimation path, but the contract does not enforce that restriction.

Since the execution mode is part of the signed call digest, a user can sign and submit an external transaction using an estimation mode. The contract will validate and execute it like any other supported mode, giving production transactions semantics intended for preflight. In particular, `EstimateGas` skips ordinary failures but reverts the whole frame on starvation, while `RaiseRevertEstimate` behaves like `RaiseRevert` with the same starvation hook.

This creates a gap between documented intent and enforced behavior. If these modes are intended only for estimation, they should not be reachable through normal externally signed execution.

Recommendation: Do not rely on `msg.sender == address(this)` alone, since an externally signed batch can self-call into the account. Instead, reject estimation modes on externally signed entrypoints, or allow them only for unsigned internal estimation frames where `opData.length == 32` and the parent execution path is already an estimation flow. If external use is intentional, document these as real production modes rather than simulation-only helpers.

LI.FI: Acknowledged. The mode is not intended to be used in on-chain but any logic we implement will sit in the hot path and waste gas. The mode itself does not introduce any critical logic but is purely a revert outcome mechanism.

Requiring msg.sender == address(this) is not safe since this can be achieve through self-call (common-pattern). As such, this would merely mask the issue not fix it.

Researcher: Acknowledged.

6.1.4 Non-EstimateGas-aware wrappers can still hide inner OOG when enough gas is retained

Context: ERC7821LIFI.sol#L199

Description: `EstimateGas` classifies a failed call as gas-starved only when the `Catapultar` frame has less than `_ESTIMATE_GAS_STARVATION_THRESHOLD` gas remaining.

This check only reflects the gas left after the immediate callee returns. If that callee is a router or wrapper, it can make an inner call that runs out of gas, catch the failure, and revert with a normal custom error while still returning more than the threshold amount of gas to `Catapultar`.

In that case, `Catapultar` treats the failure as a regular skippable business revert, even though the underlying reason was that an inner operation was gas-starved. This can allow gas estimation to settle on a value where the inner operation fails cheaply, while a higher-gas execution would run a more expensive path.

Recommendation: Document this as a known limitation of the estimation mode for arbitrary third-party routers. For high-value or unknown routes, use route-specific allowlists, trace-based checks, per-leg estimation, or conservative manual gas buffers.

LI.FI: Documented in 1ad3933

Researcher: Verified fixed documentation.

6.2 Informational

6.2.1 Starvation classification can false-positive after low-gas business reverts

Context: ERC7821LIFI.sol#L199

Description: The fixed `262,144` threshold also has a conservative failure mode: any genuine business revert that reaches Catapultar with less than the threshold remaining is classified as starvation.

This means `EstimateGas` requires every skippable failure to leave at least `262,144` gas of headroom. A late, deeply nested, or otherwise low-reserve business revert can make estimation fail even though the corresponding `SkipRevert` execution would simply log the failure and continue.

The check is also performed after Catapultar copies the full revert data and emits `CallReverted`. As a result, the gas reading does not measure the gas left immediately after the callee returned; it measures gas left after Catapultar has paid for returndata copying, memory expansion, and logging. A callee that returns a large revert payload can push the frame below the threshold during Catapultar's own failure handling, causing a normal business failure to be reported as starvation.

Recommendation: Document that estimation requires threshold headroom after skippable failures. If practical, classify starvation before unbounded returndata copying and logging in estimation modes. The sentinel check can be done with only the first 36 bytes; full revert-data logging can be deferred until after the frame is known not to be starved.

LI.FI: Acknowledged. Transactions that are so called "poison pills", including transactions failling with panic is not part of this and should be handled independently.

Researcher: Acknowledged.

6.2.2 Starvation classification is effectively untested and its docstrings overclaim

Context: ERC7821LIFI.t.sol#L198-L218

Description: Every test that claims to prove OOG-starvation detection invokes the batch with `{ gas: 250_000 }`, which is below `_ESTIMATE_GAS_STARVATION_THRESHOLD = 262_144`. At the `gas()` measurement (`ERC7821LIFI.sol#L199`) the frame therefore holds less than the threshold *unconditionally*, so *any* failure — including a plain business revert — classifies as starvation. The tests cannot distinguish a genuinely gas-starved call from a call that merely started below the threshold, and their docstrings (e.g. "a callee that wraps an inner OOG as a typed error cannot fake its gas consumption") overclaim what is actually verified.

In addition, flag 3 (`RaiseRevertEstimate`) is never exercised as a top-level mode, and neither the `0x24` size gate nor the `262_144` boundary is tested. A regression that removed the `lt(gas(), 262_144)` check, widened the size gate, or rewired flag-3 would pass the entire suite.

Recommendation: Add tests that start with ample gas (well above `64 * 262_144`) and force a *real* OOG inside the callee (e.g. an inner call given a bounded gas stipend that runs out), so the assertion isolates starvation from a low-gas start. Add top-level flag-3 coverage and boundary tests around the size gate and the threshold.

LI.FI: Fixed in 416160f

Researcher: Verified fix. The Docstrings are adjusted with more tests.

6.2.3 Solidity `README` still documents the old estimation model

Context: solidity/README.md

Description: The Solidity `README` still describes `EstimateGas` as skipping non-empty revert data and reverting on empty returndata. That is no longer how the current code works. The implementation now classifies starvation using remaining gas and re-raises `EstimateGasStarved(uint256)`.

The `README` also references `EstimateGasEmptyRevertData(bytes32)`, which is no longer present in the code, and does not document the new `0x0103...` / `RaiseRevertEstimate` mode. This is likely to mislead integrators who read the Solidity package documentation instead of the TypeScript README.

Recommendation: Update the Solidity README to describe the current gas-left threshold model, `EstimateGasStarved(uint256)`, and `RaiseRevertEstimate`. Remove references to the old empty-returndata behavior and the removed `EstimateGasEmptyRevertData(bytes32)` event.

LI.FI: Fixed: 3468bb (partially also in 1ad3933)

Researcher: Verified fix.